TECHNICAL SPECIFICATION
1-out-of-2 Deniable Plain Scalar Bit-Commitment Scheme

Proof of Concept. 128-Bit Opening Configuration

Author: Anton A. Sokolov (acmxddk@gmail.com)

License: This work is licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Abstract

Target Goal: To prove in a zero-knowledge, deniable manner that one out of two presented 256-bit scalars corresponds to a previously published public Merkle root, achieving a fixed network footprint for the challenge response phase.

Protocol Operational System Metrics

- **Post-Quantum Security Level (PQ Security):** 128-bit (NIST Level 1)
- **Cryptographic Primitive:** SHA-256 (32 bytes / 256 bits)

1 Phase 1: The Commitment Layer

1.1 Private Input and Bitstream Expansion Engine (E)

1. The Prover selects a secret 256-bit scalar ($S_{\text{committed}}$) as private input.
2. The Prover expands this secret scalar into a flat $N = 2^{10} = 1,024$-bit bitstream using a deterministic cryptographic expansion function $E(S)$.
3. The function $E(S)$ acts as a strong pseudorandom generator that shuffles the input bits exceptionally well to prevent structural analysis (for example, by utilizing sequential, labeled rounds of SHA-256 to yield exactly 1,024 bits of high-entropy output):

$$E(S) = \text{SHA256}(S \parallel \text{"Extending"} \parallel 0) \parallel \ldots \parallel \text{SHA256}(S \parallel \text{"Extending"} \parallel 3) \tag{1}$$

1.2 Flat Salt Generation

The Prover generates 1,024 unique, independent, and random 256-bit masks (salts):

$$M_0, M_1, \ldots, M_{1023} \tag{2}$$

These salts are flat, non-derived, and kept strictly private by the Prover.

1.3 Tree Construction and Public Commitment

Each leaf L_i of the commitment tree is evaluated by hashing the individual expanded bit value B_i alongside its specific 32-byte salt:

$$L_i = \text{SHA256}(B_i \parallel M_i), \quad \text{for } i \in [0 \dots 1023] \tag{3}$$

A uniform, strictly binary Merkle tree with a depth of 10 levels (1,024 leaves) is built over this layer. The resulting 32-byte Merkle Root is published to the Verifier as the public commitment.

2 Phase 2: The 1-out-of-2 Fixed Challenge Protocol

2.1 Presenting Two Scalars

During the verification phase, the Prover presents two distinct 256-bit scalars openly to the Verifier: the original true scalar $S_{\text{committed}}$ and an arbitrary alternative scalar $S_{\text{arbitrary}}$, shuffled as an unordered set $\{S_A, S_B\}$.

2.2 Challenge Derivation via Coincidence Detection

The verification routine evaluates the intersection points where both independent streams collide:

1. The Verifier computes the expansion profiles using the explicit engine: $E(S_A)$ and $E(S_B)$.
2. Because both streams are thoroughly shuffled and distributed by E, they behave as uniform pseudorandom noise. They will naturally coincide (match bit value) at an average of ~ 512 random positions:

$$\text{Coincidence Pool } C = \{i \in [0 \dots 1023] \mid E(S_A)[i] == E(S_B)[i]\} \tag{4}$$

3. Since E properly shuffles the inputs, the Prover has exactly 0 degrees of freedom to manipulate or cheat with the intersection positions. The positions are completely locked by the deterministic outputs of E.

2.3 Fixed 128-Bit Opening Sequence

To minimize packet size and stabilize network traffic, the protocol cuts off the response phase at the security boundary:

1. The Verifier selects deterministically, looking at both extended streams, exactly 128 indices from the derived Coincidence Pool C to form a fixed Spot-Check Challenge.
2. **The Proof Payload:** The Prover responds by sending only the 128 explicit 256-bit masks (M_i) corresponding strictly to these 128 selected indices, alongside the required Merkle Multi-proof companion hashes.
3. **Verification Check:** The Verifier reads the shared bit values ($B_i^A == B_i^B$) for these 128 indices, computes the leaves $L_i = \text{SHA256}(B_i \parallel M_i)$, and processes them against the public Merkle Root according to one of the options defined in Section 4.

3 Security and Traffic Metrics

3.1 Post-Quantum Soundness and Deniability

- **The Soundness Floor:** Since passing each checked index requires guessing a hidden salt, a cheating Prover has a 0.5 chance of passing a single leaf test. By opening exactly 128 positions, a soundness error of exactly $(0.5)^{128} = \mathbf{2^{-128}}$ is reached. This 2^{-128} confidence threshold is always guaranteed, regardless of how many additional matching elements exist in the broader pool.
- **Zero-Knowledge Privacy:** Because the evaluated indices match identically for both inputs, the proof is perfectly symmetric. The Verifier is convinced that one of the scalars is valid, but receives exactly zero information pointing to which one was committed.

4 Verification Options and Proof Optimization

The software test suite implements two processing frameworks for proof generation and multi-proof evaluation. The choice of strategy directly dictating the final network packet overhead.

4.1 Option A: Unoptimized Naive Layout

The naive approach processes each selected challenge index independently, tracing it standalone from the leaf up to the root level.

- **Theoretical Complexity:** By iterating over a fixed sample size of $k = 128$ leaves across a tree depth of $d = 10$, the engine duplicates overlapping sibling paths across higher branch spaces. This introduces an empirical average of $\sim 1,280$ companion path hashes.
- **Proof Footprint Metrics:**

$$\begin{aligned} \text{Masks Segment: } & 128 \times 32 \text{ bytes} = 4,096 \text{ bytes} \\ \text{Path Hashes: } & \sim 128 \times 10 \times 32 \text{ bytes} \approx 40,960 \text{ bytes} \\ \text{Total Network Data: } & \sim \mathbf{45,056 \text{ bytes}} \ (\sim 44.0 \ \mathbf{KB}) \end{aligned}$$

4.2 Option B: Optimized Octopus Multi-Proof Engine

The Octopus configuration treats the $k = 128$ opened challenge indices as a single unified layer, grouping parent hashes level-by-level up the tree structure.

- **Theoretical Optimization:** When evaluating a large sparse volume of indices ($k = 128$) distributed uniformly across a small tree ($N = 1024$), neighboring paths naturally merge. The Octopus engine programmatically prunes out any structural internal node hash that can be inferred or calculated by combining adjacent open data cells. This bounds the total unique sibling path requirements to an empirical average of only ~ 120 unique hashes.
- **Proof Footprint Metrics:**

$$\begin{aligned} \text{Masks Segment: } & 128 \times 32 \text{ bytes} = 4,096 \text{ bytes} \\ \text{Path Hashes: } & \sim 120 \times 32 \text{ bytes} = 3,840 \text{ bytes} \\ \text{Total Network Data: } & \mathbf{7,936 \text{ bytes}} \ (\sim \mathbf{7.75 \ KB}) \end{aligned}$$

4.3 Performance Summary Matrix

Verification Strategy	**Avg. Companion Hashes**	**Total Payload Weight**	**Network Footprint**
Unoptimized Naive Mode	$\sim 1,280$	$\sim 45,056$ bytes	~ 44.00 KB
Optimized Octopus Mode	~ 120	$\mathbf{7,936}$ **bytes**	$\sim$ **7.75 KB**

Table 1: Cryptographic Proof Layout Footprint Metrics.